INTERNATIONAL CYBERSECURITY AND DIGITAL FORENSICS ACADEMY

FORMAL DIGITAL FORENSICS INVESTIGATION REPORT

SBT-DF203 - LAB 6: Firewall Traffic Control and Forensic Verification

Field	Value
Case Identifier	SBT-DF203-Lab6
Lead Examiner	Nebeuwa Ifeanyichukwu Raphael
Registration / ID No.	202520850LE
Course Code	SBT-DF203
Course Title	Basic Networking Skills for Digital Forensics
Lab Number	Lab 6
Lab Title	Firewall Traffic Control and Forensic Verification
Date of Submission	September 18, 2026
Primary Target System	Kali Linux Workstation (kali@kali) with isolated Windows 10 VM
Evidence Type	HTTP PCAPNG captures, iptables ruleset exports and command-output artifacts

Prepared for authorized academic forensic examination

1. Executive Summary

This report documents my performance of SBT-DF203 Lab 6, a firewall traffic-control and forensic-verification exercise conducted in an isolated VMware environment. The practical established a Kali Linux web server, verified the lab network, preserved the original firewall state, captured an allowed HTTP baseline, inserted a narrowly scoped iptables DROP rule for the Windows client, captured the resulting blocked connection behaviour, compared the allowed and blocked packet traces, and removed the rule to restore access.

The final lab network used Kali Linux at 192.168.190.129/24 and Windows 10 at 192.168.190.128/24 on the 192.168.190.0/24 isolated network. Apache listened on TCP port 80. The baseline HTTP test completed successfully with HTTP 200 OK responses. The allowed capture contained the normal TCP three-way handshake and HTTP request/response traffic.

After the DROP rule was applied to source 192.168.190.128 and destination TCP port 80, the blocked capture contained repeated SYN transmissions without a corresponding SYN-ACK. The blocked trace contained five captured SYN packets, with retransmission indicators on the subsequent attempts. This behaviour is consistent with silent packet dropping. After the exact rule was deleted, Windows again connected successfully and received HTTP 200 OK.

A small operational issue occurred when a root-owned TShark process attempted to write directly into the evidence directory. The capture was instead written to /tmp and moved into evidence/, after which its permissions were corrected for analysis. This troubleshooting step is documented rather than omitted.

2. Investigation Overview and Scope

2.1 Purpose and Objectives

- Explain the purpose of host-based firewall control and the role of iptables chains.
- Record the network state and establish a successful HTTP baseline before modifying the firewall.
- Preserve the original firewall state and associated SHA-256 evidence.
- Create, verify and later remove a source-specific TCP/80 DROP rule.
- Capture and compare allowed and blocked HTTP connection attempts.
- Correlate packet-level behaviour with firewall operation and client application results.
- Explain DROP versus REJECT and identify evidential limitations.
- Restore the firewall state and confirm that HTTP access returned.

2.2 Environment and Evidence

Component	Observed configuration
Kali Linux	192.168.190.129/24, eth0
Windows 10	192.168.190.128/24, Ethernet0
Network	Isolated 192.168.190.0/24 VMware virtual network
Web server	Apache2 on TCP port 80
Firewall	iptables
Capture/analysis	TShark/Wireshark
Client testing	curl from Windows 10

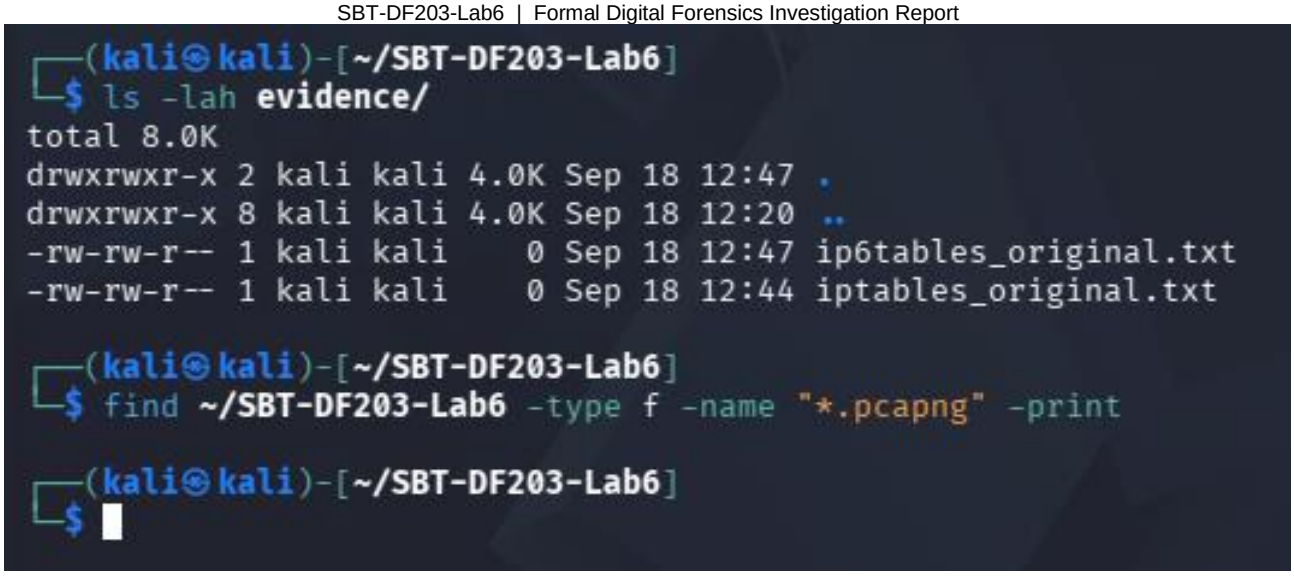


Figure 1: Lab 6 workspace directories created under ~/SBT-DF203-Lab6.

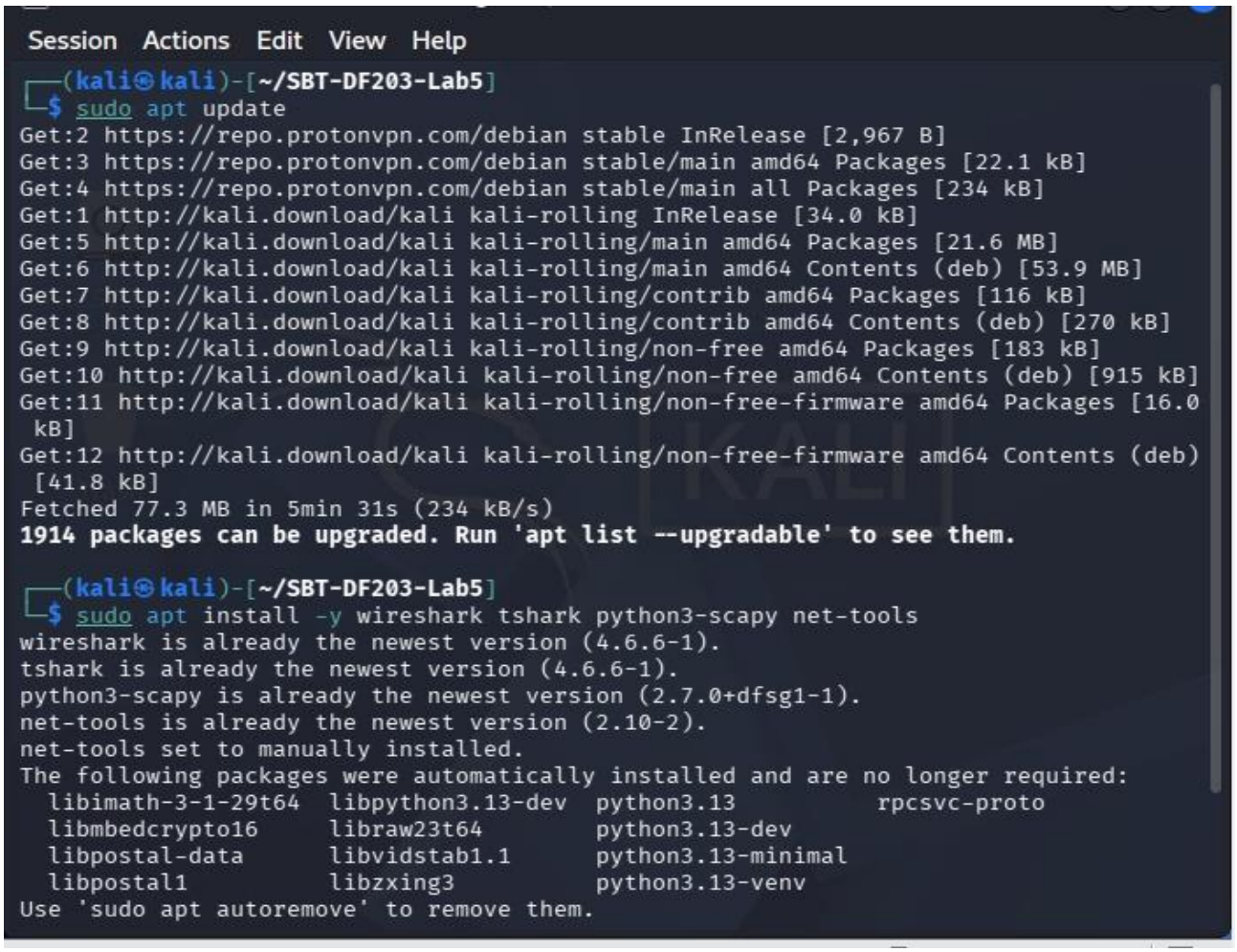


Figure 2: Kali package preparation showing apt update and installation of Apache2, curl, iptables, TShark and Wireshark.

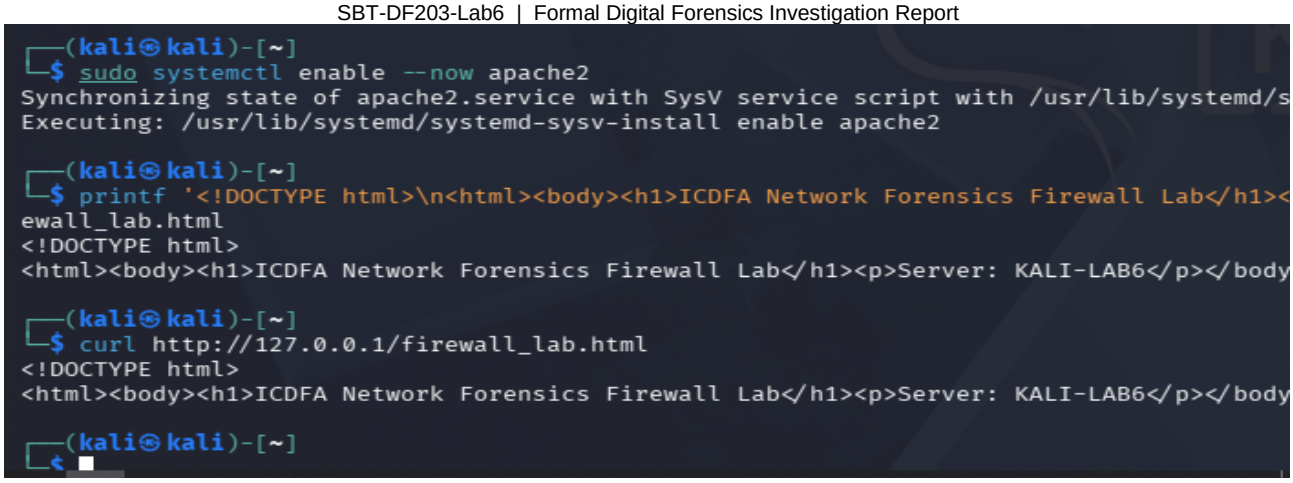


Figure 3: Apache2 enabled and the training firewall_lab.html page created and retrieved locally.

3. Evidence Preservation and Original Firewall State

The Lab 6 instructions require the existing firewall ruleset to be exported before making changes. The original INPUT, FORWARD and OUTPUT chains were recorded before the source-specific DROP rule was inserted. The captured original ruleset was empty of user-defined rules, with the default policy set to ACCEPT.

The SHA-256 value recorded for the empty original ruleset file was e3b0c44298fc1c149afbf4c8996fb92427ae41e4649b934ca495991b7852b855. This is the standard SHA-256 digest of an empty file and was the value returned by the sha256sum command in the lab session.

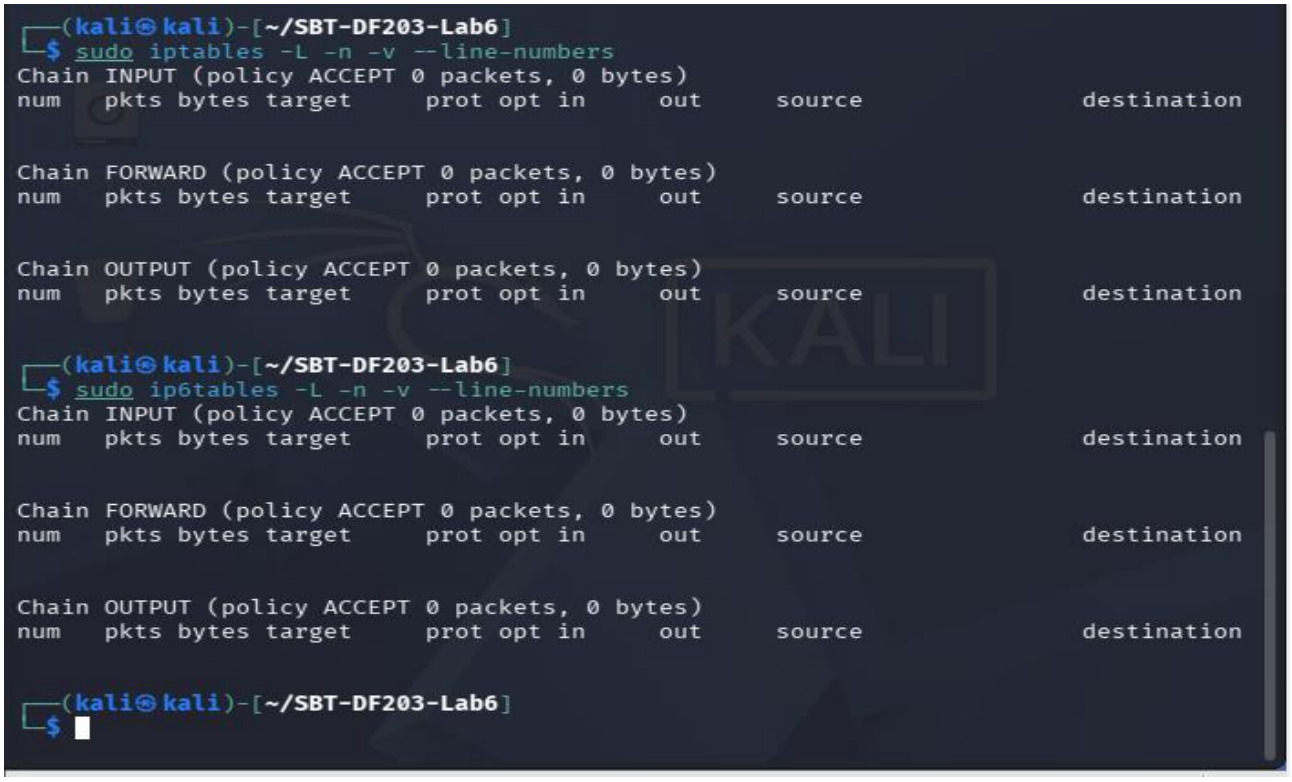


Figure 4: Original INPUT, FORWARD and OUTPUT chains before the blocking rule was added.

3.1 Network and Service Baseline

The final network configuration showed Kali at 192.168.190.129/24 on eth0 with a connected route for 192.168.190.0/24. Windows reported 192.168.190.128/24 with a /24 mask. Apache was enabled during setup and the training page was served from /firewall_lab.html.

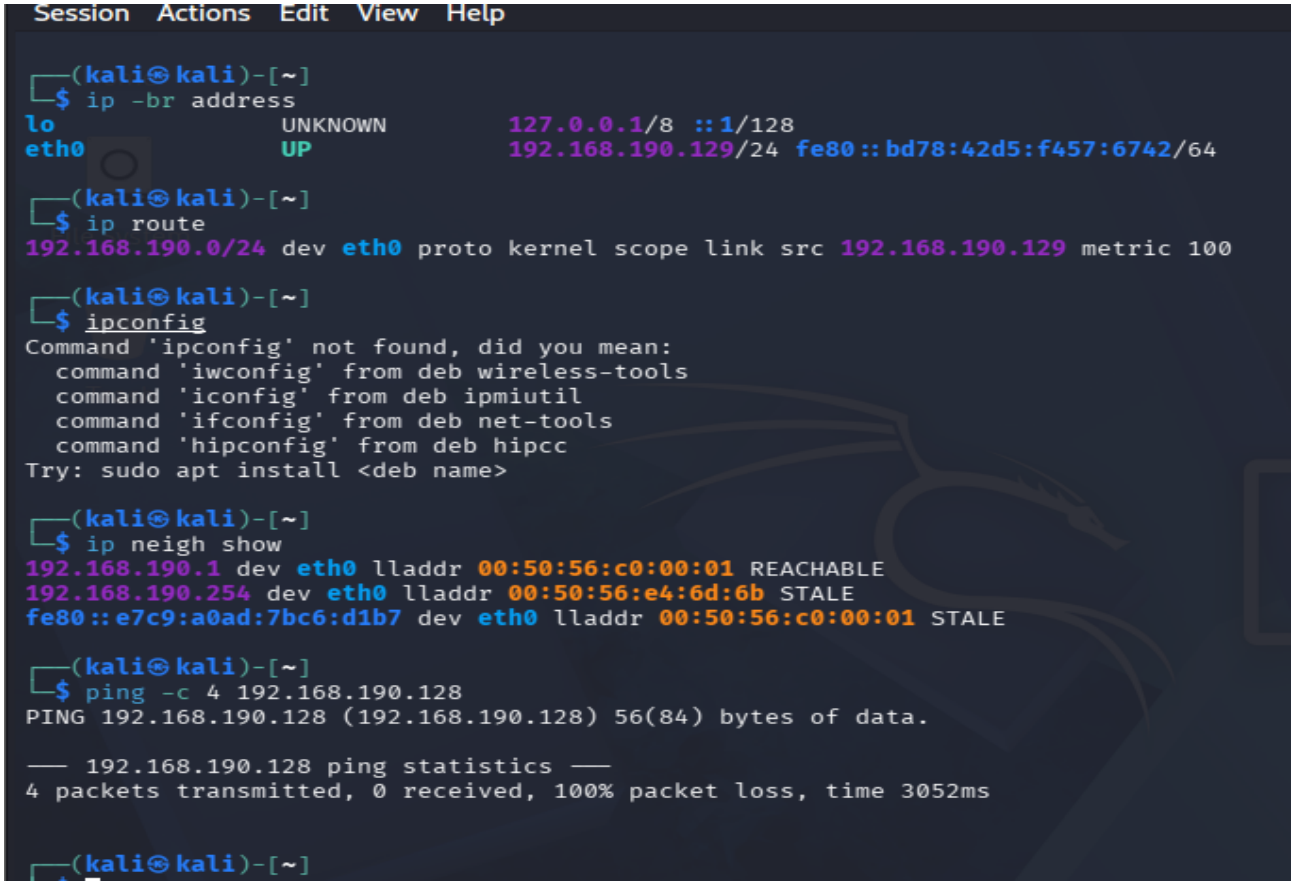


Figure 5: Kali final network configuration showing eth0 at 192.168.190.129/24 and the connected 192.168.190.0/24 route.

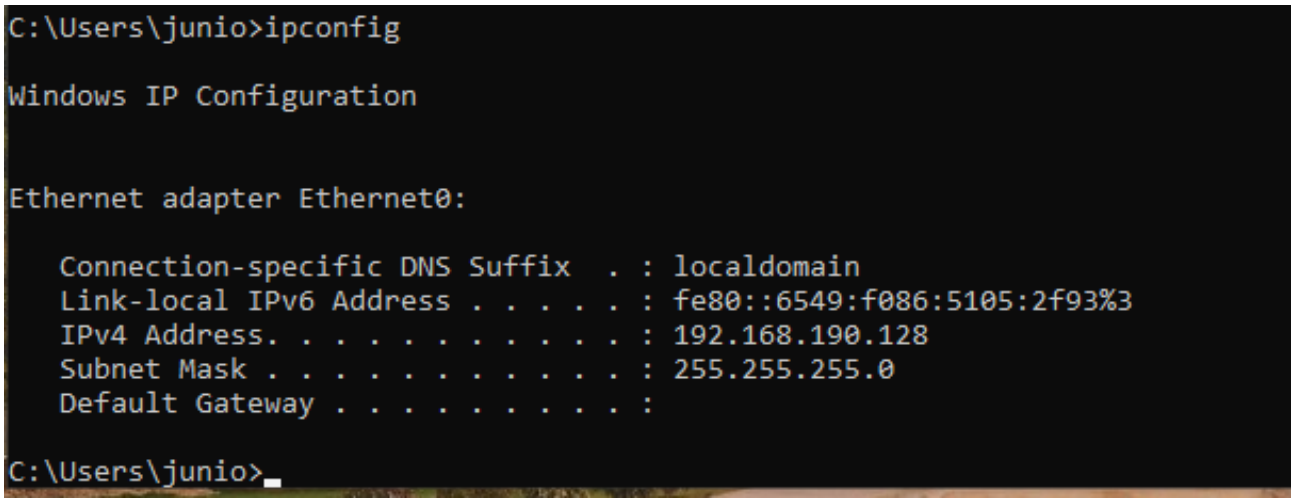


Figure 6: Windows 10 baseline IPv4 configuration showing 192.168.190.128/24.

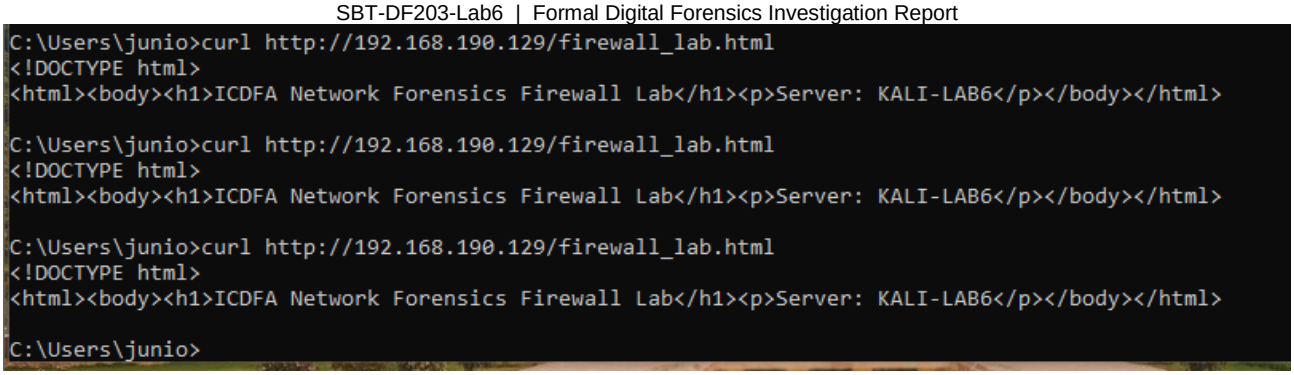


Figure 7: Successful baseline curl requests from Windows 10 to the Kali Apache service.

4. Part A - Record the Lab Network and Baseline Access

The baseline test established that the client could reach the Kali web server before firewall modification. Windows 10 used curl to request http://192.168.190.129/firewall_lab.html. The response contained HTTP/1.1 200 OK and the expected ICDFA Network Forensics Firewall Lab HTML content.

This baseline is important because it demonstrates that the web service was functioning before the blocking rule was introduced. It provides a control condition for interpreting the later blocked connection attempts.

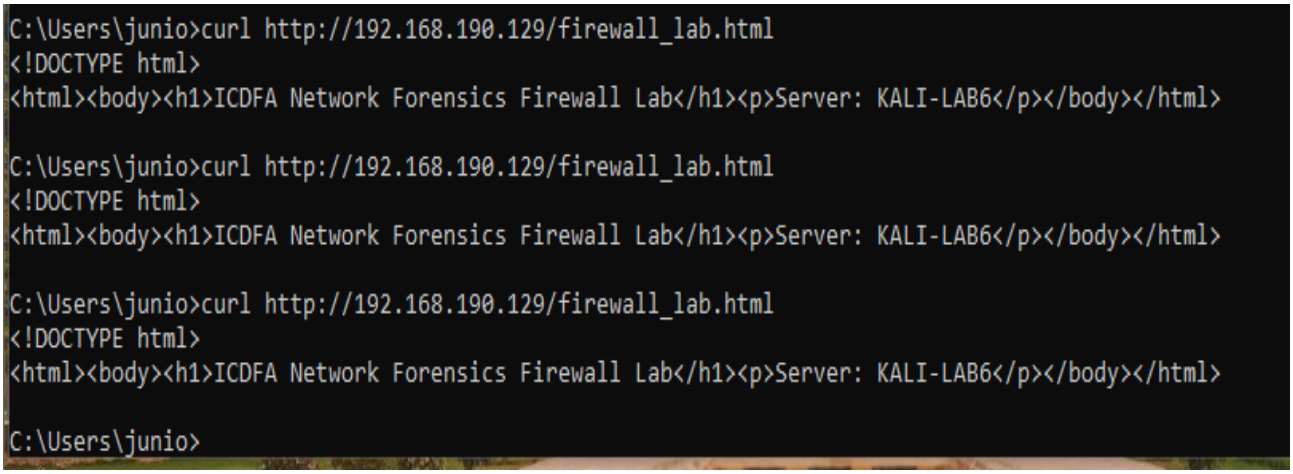


Figure 7: Successful baseline curl from Windows 10 to the Kali Apache service.

5. Part B - Capture the Allowed HTTP Baseline

TShark was used on eth0 with a TCP port 80 filter for the client/server pair. During the capture, Windows generated HTTP requests for /firewall_lab.html. The successful trace contained the TCP handshake, HTTP GET request, HTTP 200 response and normal TCP termination. The capture reported 18 packets.

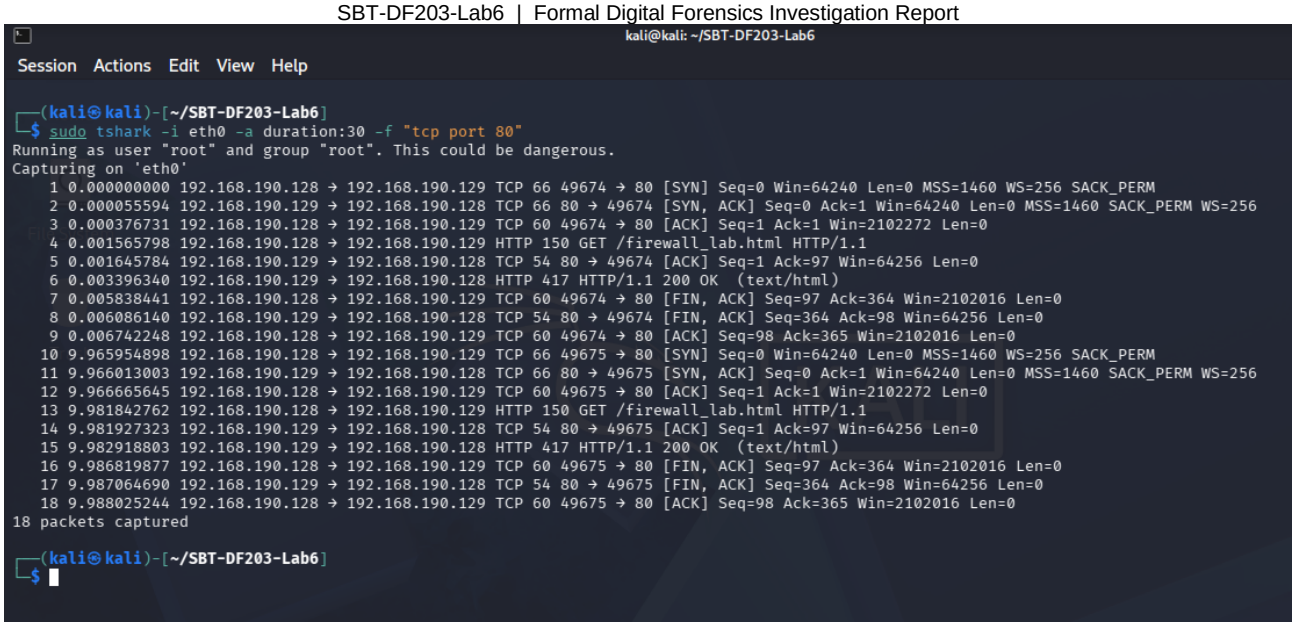


Figure 8: Allowed HTTP baseline capture showing TCP handshake, HTTP GET /firewall_lab.html and HTTP 200 OK responses; 18 packets were captured.

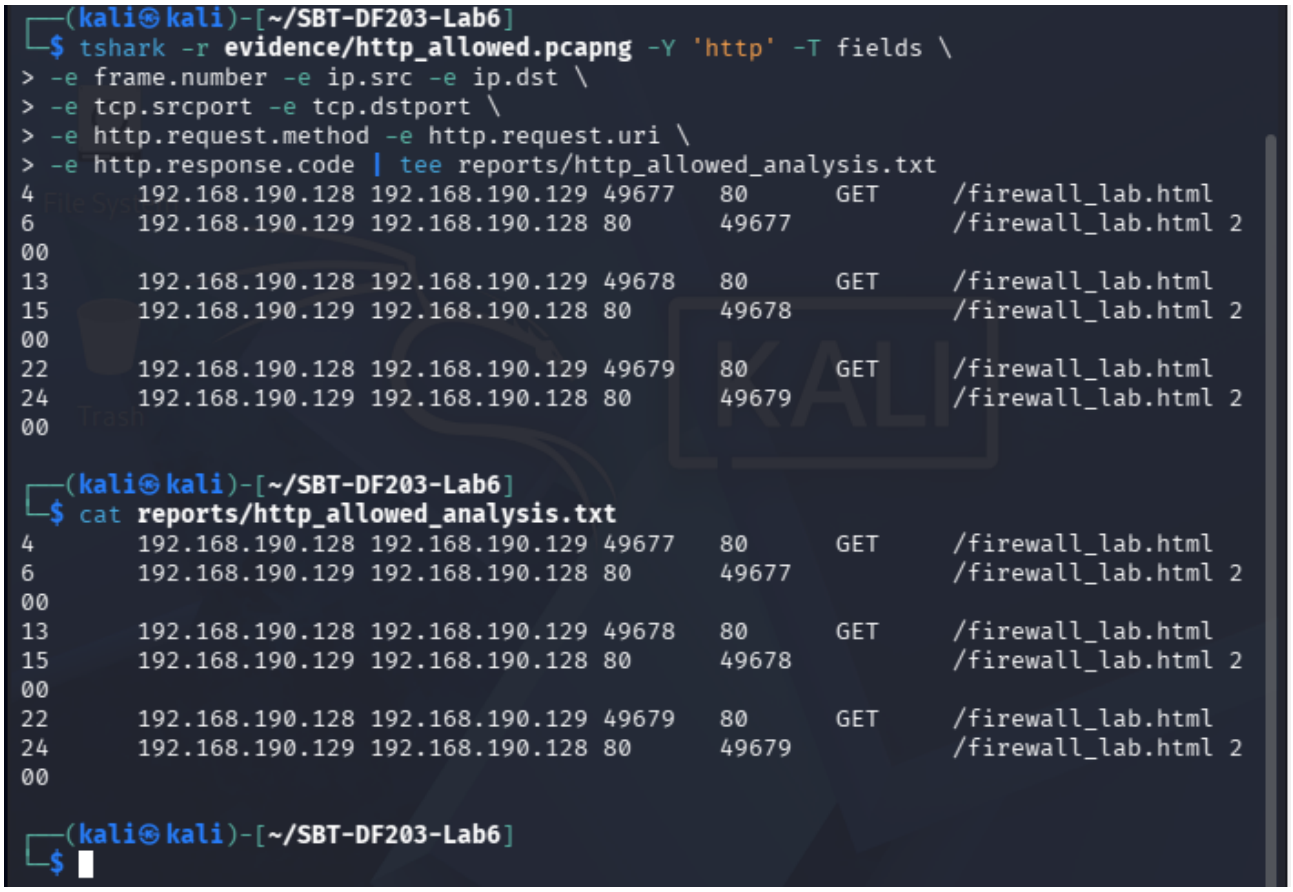


Figure 9: TShark analysis of the allowed HTTP capture showing three successful HTTP request/response exchanges.

The extracted allowed-session evidence shows that the server replied to the client's SYN, the TCP handshake completed, the HTTP request reached Apache, and the server returned a 200 response. These observations establish the expected packet-level baseline against which the blocked trace can be compared.

6. Part C - Apply and Verify the Blocking Rule

The blocking rule was limited to the assigned Windows client and TCP destination port 80. The rule specification used was: iptables -I INPUT 1 -s 192.168.190.128 -p tcp --dport 80 -j DROP. The rule was then checked with iptables -C and the session recorded 'Rule verified present.'

Rule element	Value
Chain	INPUT
Source IP	192.168.190.128
Protocol	TCP
Destination port	80
Action	DROP
Position	Rule 1

The rule was intentionally narrow rather than a blanket firewall flush. This preserved the surrounding firewall state and allowed the later cleanup command to remove the exact rule specification.

7. Part D - Capture Blocked Traffic and Rule Counters

The first attempt to save the blocked capture directly under evidence/ produced a permission-denied error because TShark was running with elevated privileges while the destination path was not writable in the resulting process context. The capture was therefore written to /tmp/http_blocked.pcapng and then moved into evidence/. The file was subsequently changed to mode 644 so it could be analyzed without root privileges.

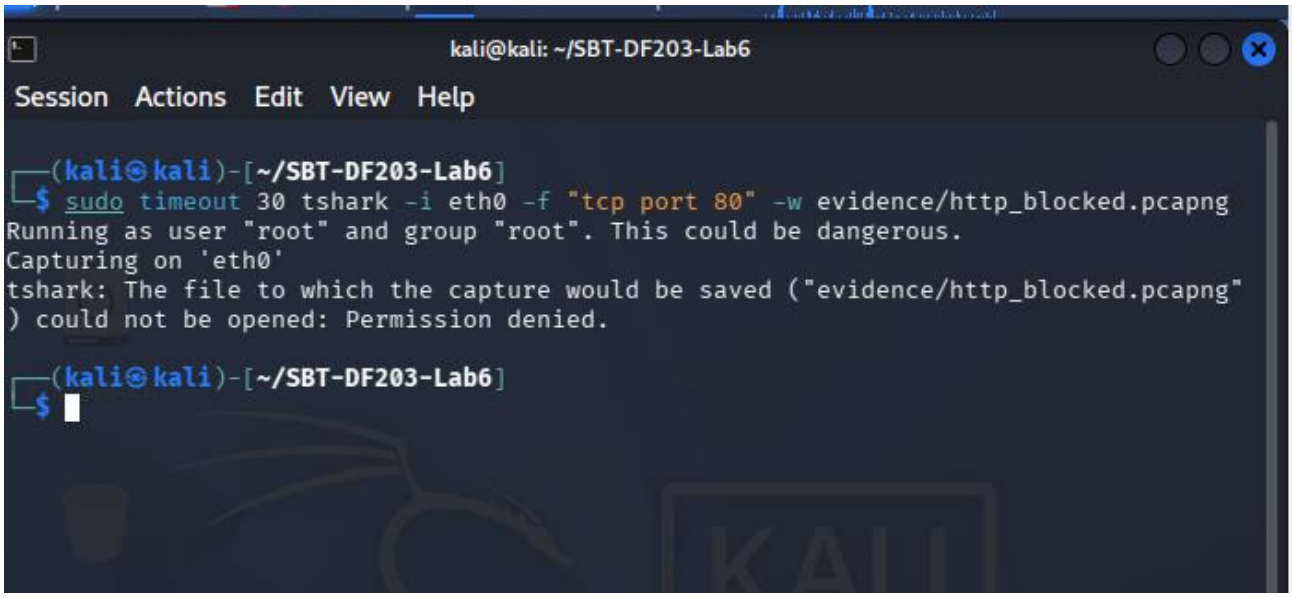


Figure 10: Initial blocked-capture attempt showing a file-permission problem when writing directly into evidence/.

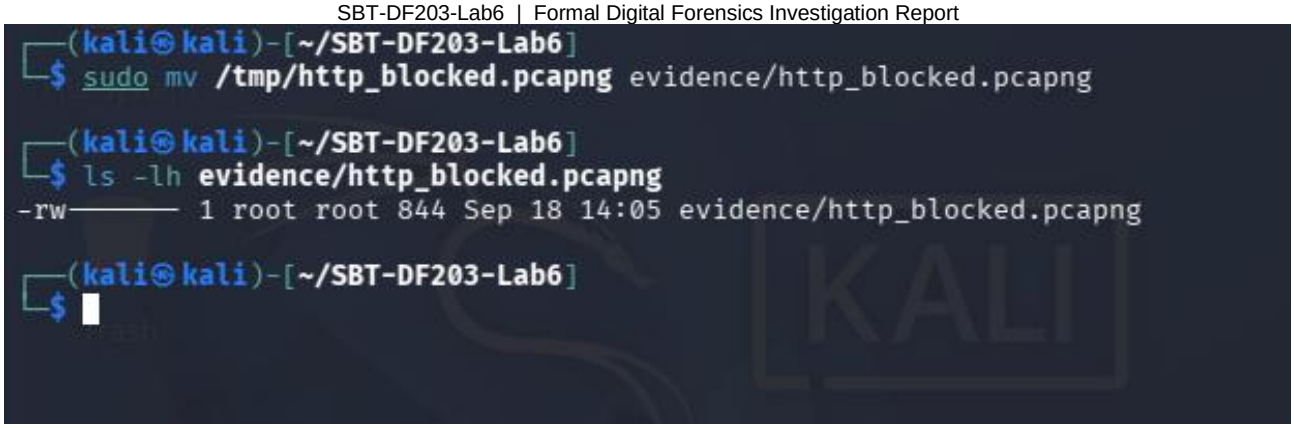


Figure 11: Blocked capture successfully moved into evidence/http_blocked.pcapng and verified as an 844-byte capture.

The final blocked capture contained five TCP SYN packets from 192.168.190.128 to 192.168.190.129:80. The first SYN initiated the connection attempt, while the later SYNs were retransmissions. No SYN-ACK from the server and no HTTP request/response exchange were observed in this blocked trace.

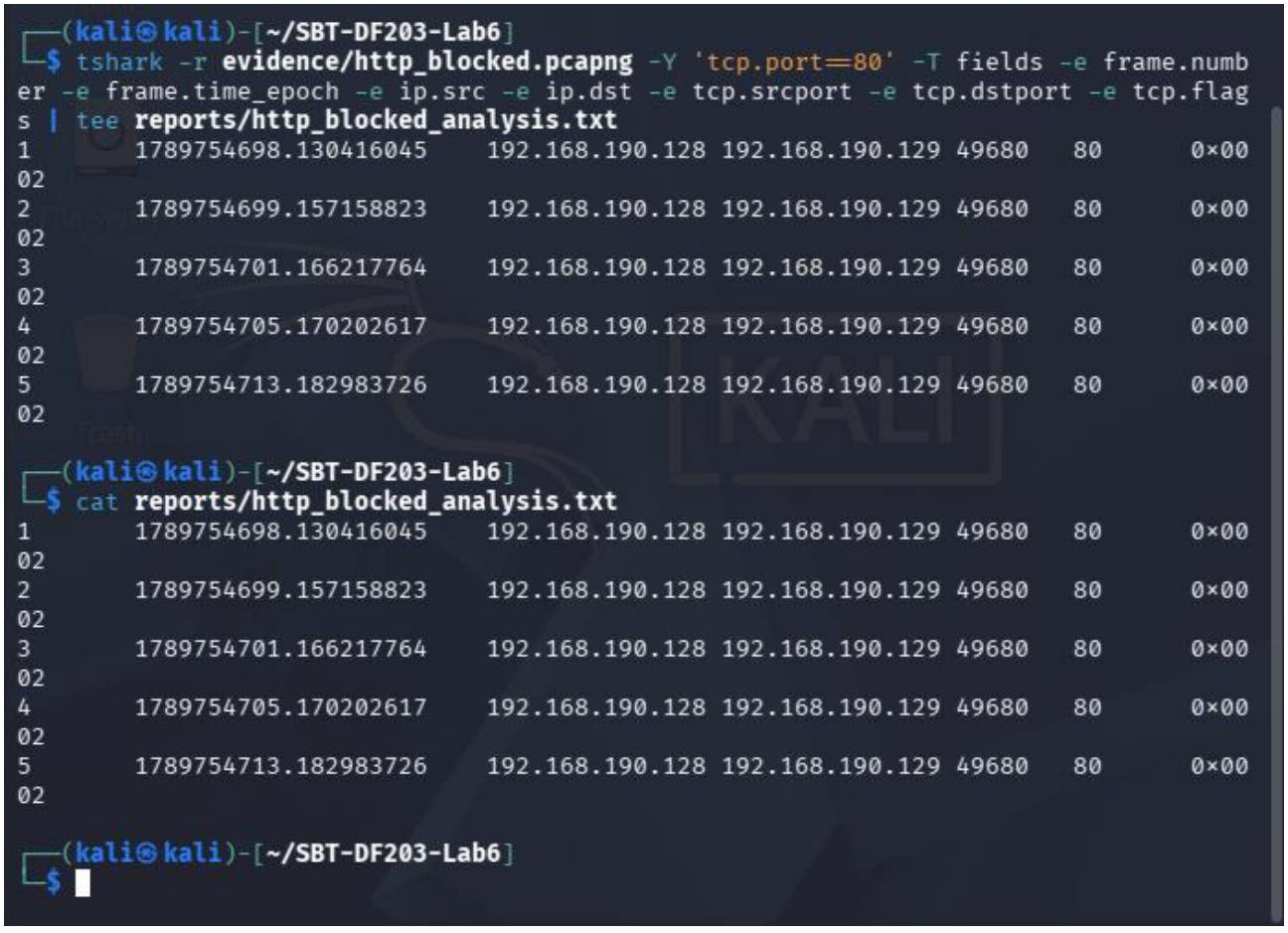


Figure 12: Blocked HTTP capture analysis showing repeated SYN packets from 192.168.190.128 to port 80 with retransmission indicators and no server response.

The packet evidence is consistent with a DROP action: the firewall silently discards matching traffic, so the client does not receive the SYN-ACK required to complete the TCP handshake and continues retransmitting until its connection attempt times out. The lab also requires firewall counters to be recorded after the blocked test. The command used for this purpose was sudo iptables -L INPUT -n -v --line-numbers; the exact post-test per-rule counter value is not reproduced here because it was not preserved in the available screenshot set.

8. Part E - Compare Allowed and Blocked Captures

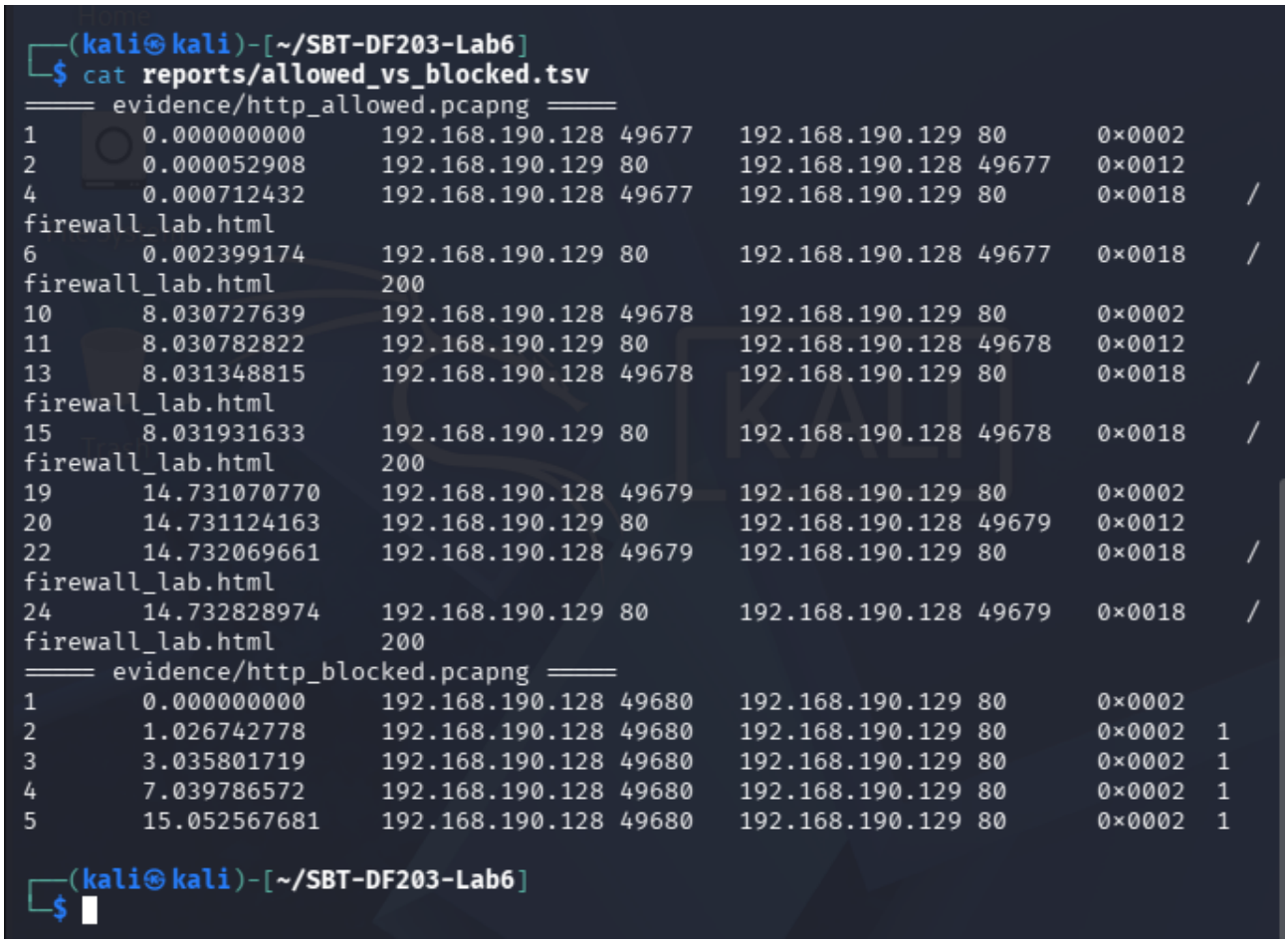


Figure 13: Allowed-versus-blocked TShark comparison; the allowed capture contains handshake/HTTP responses while the blocked capture contains repeated SYN retransmissions only.

Indicator	Allowed capture	Blocked capture
Client SYN visible?	Yes	Yes
Server SYN-ACK visible?	Yes	No
Handshake completed?	Yes	No
HTTP GET visible?	Yes	No
HTTP response visible?	Yes, HTTP 200	No
Retransmissions/timeouts	No abnormal retransmission pattern	Repeated SYN retransmissions
Firewall effect	Traffic passed	Matching TCP/80 traffic dropped
curl result	Successful HTTP response	Connection attempt failed/timed out

The comparison provides the central forensic correlation for the exercise. In the allowed trace, the network and application layers both show successful communication. In the blocked trace, the client-originated SYN is visible but the server's SYN-ACK is absent, the HTTP layer is never reached, and the client continues retransmitting. The difference is therefore observable at both packet and application levels.

9. Part F - Remove the Rule and Restore Access

The exact blocking rule was deleted using the same source, protocol and destination-port specification used when it was inserted. The subsequent iptables -C check returned an error indicating that no matching rule remained, followed by 'Rule successfully removed'. The restored INPUT chain showed the ACCEPT policy with no blocking rule listed.

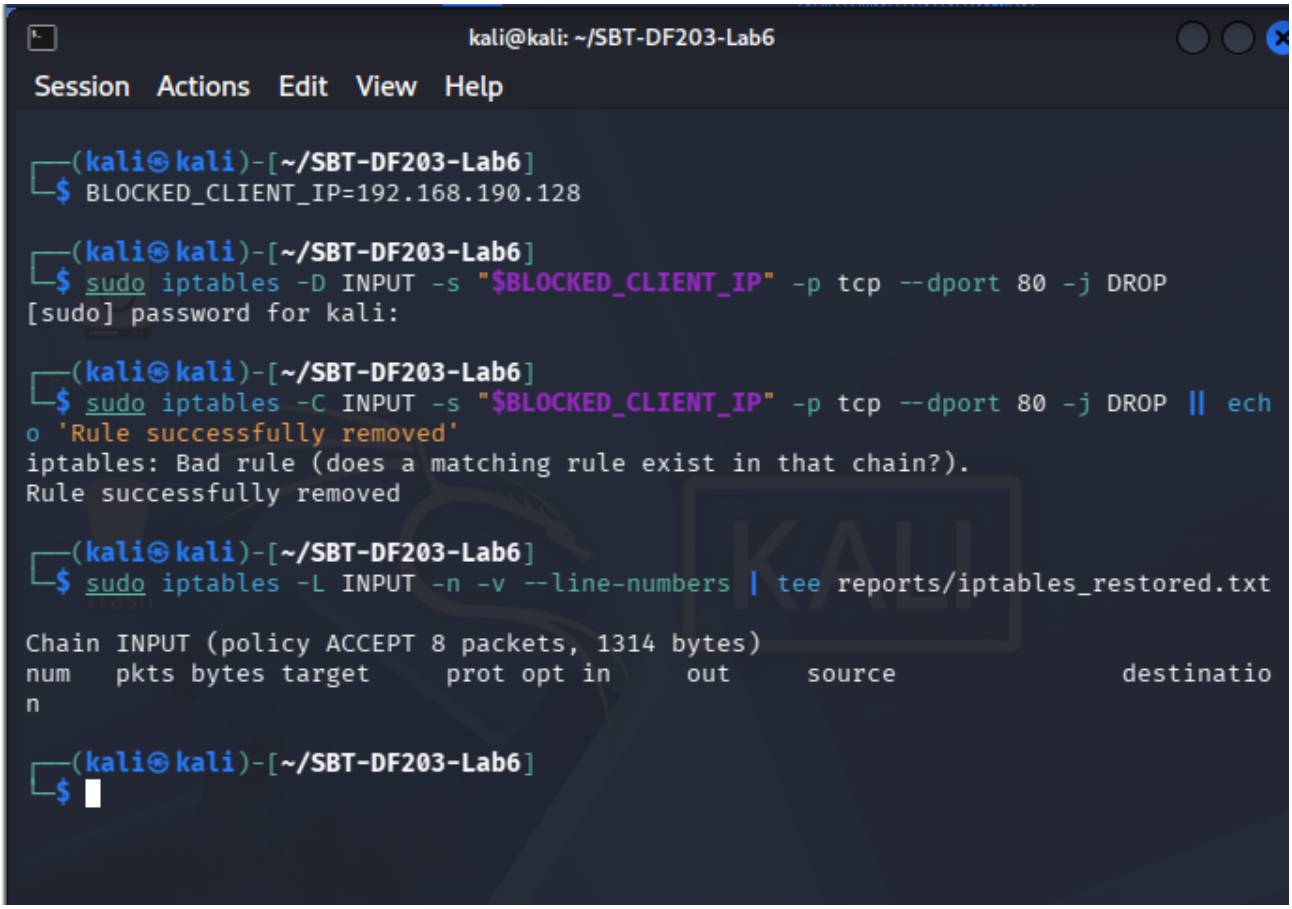


Figure 14: Exact DROP rule removal, verification that the rule no longer exists, and restored INPUT policy.

A final Windows curl request to http://192.168.190.129/firewall_lab.html then succeeded. The client connected to port 80, issued GET /firewall_lab.html, and received HTTP/1.1 200 OK with the expected lab HTML. This restored-access test provides direct confirmation that the service remained available and that the source-specific firewall rule had been removed.

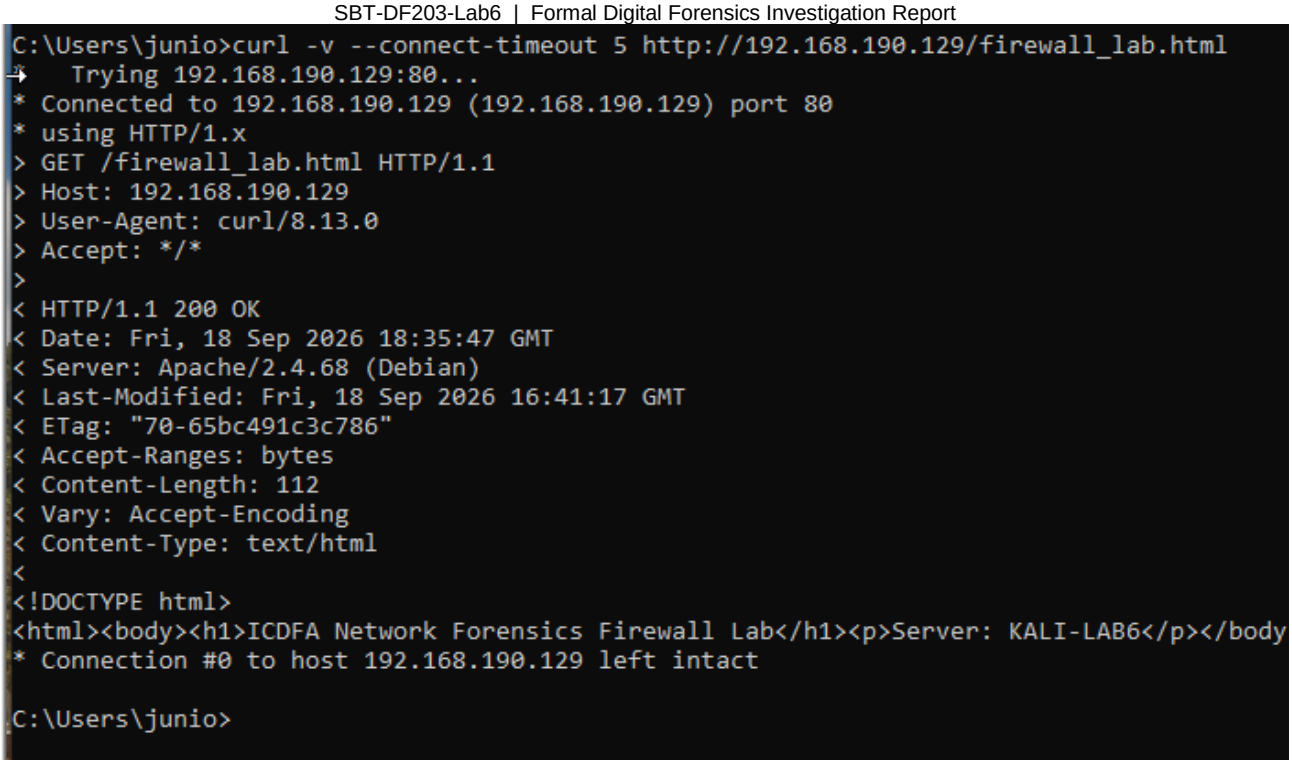


Figure 15: Windows 10 successful HTTP access after cleanup, returning HTTP/1.1 200 OK and the expected lab HTML.

10. DROP Versus REJECT Discussion

DROP and REJECT differ in how the firewall handles matching traffic. DROP silently discards the packet. For a TCP SYN, the client receives no SYN-ACK and therefore commonly retransmits the SYN before eventually timing out. The blocked capture in this lab demonstrates this pattern.

REJECT actively informs the sender that the traffic is not being accepted. Depending on the rule and protocol, the response may be a TCP reset or an ICMP error. Consequently, the client normally receives an explicit failure sooner, and the packet capture contains evidence of the rejection rather than only repeated unanswered SYNs.

11. Forensic Interpretation Questions - Question 15

1. **Why does DROP commonly cause SYN retransmissions and a timeout?**
   A TCP connection begins with a SYN from the client. When a firewall DROP rule silently discards that SYN, the server does not receive it and therefore does not send the SYN-ACK that the client expects. The client retransmits the SYN after its retransmission timer expires. If no response arrives, the connection attempt eventually times out.
2. **How would a REJECT rule differ in the packet capture and client output?**
   REJECT actively sends a failure response instead of silently discarding the packet. Depending on the rule, the capture may show a TCP RST or an ICMP error. Because the client receives an explicit response, curl normally reports a faster connection failure rather than waiting through repeated SYN retransmissions.
3. **Why are firewall counters valuable corroborating evidence?**
   iptables counters record the packets and bytes matched by a rule. If the counter increases during a controlled test from the known client, it provides independent corroboration that the firewall rule

actually matched the traffic. The counter should be interpreted together with packet-capture and application evidence rather than treated as the only proof.

4 **What evidence would show that the web server was down rather than firewall-blocked?**

Useful evidence would include the absence of a listening socket in ss -lntp, an inactive or failed Apache service status, local failure when testing the service on the server itself, and appropriate Apache/service logs. In contrast, a firewall-blocking explanation is strengthened when the server is listening, the client sends SYNs, the server does not respond, and the relevant firewall rule counters increase.

5 **What is the risk of deleting a rule by line number after other rules have changed?**

Rule numbers can change when rules are inserted or deleted. A line-number deletion performed after such changes can therefore remove the wrong rule. Using the exact rule specification with iptables -D, followed by iptables -C to verify that the intended rule is absent, reduces this risk.

6 12. Evidence Inventory and Reproducibility

Evidence / Report Artifact	Role / observation
evidence/iptables_original.txt	Original firewall ruleset export before the DROP rule.
reports/iptables_original_hash.txt	SHA-256 record; empty-file digest e3b0c44298fc1c149afbf4c8996fb92427ae41e4649b934ca495991b7852b855.
evidence/http_allowed.pcapng	Allowed HTTP baseline capture; successful TCP/HTTP session.
reports/http_allowed_analysis.txt	TShark field extraction for the allowed capture.
evidence/http_blocked.pcapng	Blocked HTTP capture; 5 SYN packets including retransmissions.
reports/http_blocked_analysis.txt	TShark analysis showing repeated SYN attempts and no response.
reports/allowed_vs_blocked.tsv	Comparative packet-level analysis.
reports/rule_verification.txt	Confirmation that the source-specific DROP rule was present.
reports/iptables_restored.txt	Firewall state after the exact rule was removed.

13. Required Forensic Findings

Finding	Observed result
Network configuration	Kali 192.168.190.129/24; Windows 192.168.190.128/24; same /24 lab network.
Baseline service	Apache HTTP service reachable on TCP/80; Windows curl received HTTP 200 OK.
Allowed capture	18 packets; TCP handshake and HTTP GET/200 response observed.
Blocking rule	INPUT rule for source 192.168.190.128, TCP destination port 80, target DROP.
Blocked capture	5 SYN packets; later SYNs marked as retransmissions; no SYN-ACK or HTTP response.
Comparison	Allowed traffic completed TCP/HTTP exchange; blocked traffic stopped at connection establishment.
Cleanup	Exact rule removed and verification confirmed it was absent.
Restored access	Windows curl again received HTTP 200 OK after cleanup.

14. Conclusions and Appendix

The practical established a reproducible firewall-control experiment in an isolated environment. The allowed capture demonstrated normal TCP and HTTP behaviour, while the blocked capture showed the characteristic evidence of a silently dropped TCP connection: repeated client SYNs without a server SYN-ACK or HTTP exchange. The successful post-cleanup curl test demonstrated restoration of access.

The evidence supports the conclusion that the source-specific DROP rule affected the Windows client's TCP/80 traffic during the controlled test. The report distinguishes direct packet observations from interpretation and records the temporary file-permission troubleshooting encountered during capture.

Appendix A - Key Commands

```
sudo iptables-save | tee evidence/iptables_original.txt

sudo iptables -L -n -v --line-numbers

ip -br address

ip route

sudo ss -lntp | grep ':80'

curl -v --connect-timeout 5 http://192.168.190.129/firewall_lab.html

sudo tshark -i eth0 -f 'tcp port 80' -a duration:30 -w
evidence/http_allowed.pcapng

sudo iptables -I INPUT 1 -s 192.168.190.128 -p tcp --dport 80 -j DROP

sudo iptables -C INPUT -s 192.168.190.128 -p tcp --dport 80 -j DROP

sudo tshark -r evidence/http_blocked.pcapng -Y 'tcp.port==80' -T fields ...

sudo iptables -D INPUT -s 192.168.190.128 -p tcp --dport 80 -j DROP
```

Appendix B - Academic Integrity and Authorization Statement

I confirm that this lab was completed in an authorized isolated environment, that the firewall was changed only for the controlled exercise, that the original firewall state was recorded before modification, and that the final blocking rule was removed before the practical was concluded. The screenshots and observations in this report are based on the recorded Lab 6 session evidence.